

Abstract Base class & @ abstract method

NOTES

You Can't make objects for this abstract based class

Abstract classes:-

Q. What is an Abstract class?

- A class that cannot be instantiated directly.
- Used to define a common interface for all its subclasses.
- Created using the abc module (ABC class).

Q. Why use it?

- Ensures that subclasses implement specific methods.
- Promotes Code Consistency and reusability.

Q. How to Create one:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
```

```
    def make_sound(self):
```

```
        pass
```

- Animal is abstract
- make_sound() is an abstract method - must be defined in child classes.

Subclass eg:-

```
class Dog(Animal):
```

```
    def make_sound(self):
```

```
        return "Bark"
```

- ✓ Dog implements make_sound(), so it can be instantiated
- ✗ if make_sound() is missing, python will raise an error

◆ Key points:

- Use ABC as base class
- Decorate required methods with @ abstractmethod
- Abstract classes help enforce method contracts across subclasses.

SUMMARY

Getters and setters in python

4-2-2026

NOTES

In python, a getter and setter are methods used to access and update the attributes of a class. These methods provide a way to define controlled access to the attributes of an object, thereby ensuring the integrity of the data.

→ By default attributes in python can be accessed directly.

→ But this may pose problems when attributes need validation or transformation before being assigned or retrieved.

→ Getter:- The getter method is used to retrieve the value of a private attribute

- It allows controlled access to the attribute

→ Setter:- The setter method is used to set or modify the value of a private attribute.

- It allows us to control how the value is updated, enabling validation (or) modification of the data before it's actually assigned.

→ Python provides several ways to implement getter, setter methods:

1. Using normal function:

In this approach getter & setter methods are explicitly defined to get and set the value of a private variable.

The setter allows setting the value and the getter is used to retrieve it.

class Greeter:

```
def __init__(self, age=0):
```

```
    self.age = age
```

#getter method

```
def get_age(self):
```

```
    return self._age
```

Setter method

```
def set_age(self, x):
```

```
    self._age = x
```

```
Raj = Greeter()
```

#Setting the age using setter

```
Raj.set_age(21)
```

retrieving age using getter

```
Print(Raj.get_age()) // 21
```

```
Print(Raj._age) // 21
```

._age is a way to create a private attribute

NOTES

Explanation:

- A Greek class is defined with an `_age` attribute
- The `get_age()` method is the getter, which retrieves the value of `_age`
- The `set_age()` method is the setter, which assigns a value to `_age`
- The setter method is used to set the age and the getter method is used to retrieve it.

2. Using `property()` function:

→ In this method, the `property()` function is used to wrap the getter, setter and deleter methods for an attribute, providing a more streamlined approach.

Class Greeks:

```
def __init__(self):
    self._age = 0
```

→ A Greek class is created with an internal `_age` attribute

function to get value of `_age`

```
def get_age(self):
```

```
    print("getter method called")
```

```
    return self._age
```

→ The `get_age()` function retrieves the value

function to set value of `_age`

```
def set_age(self, a):
```

```
    print("setter method called")
```

```
    self._age = a
```

→ # This sets the variable `_age`

function to delete `_age` attribute

```
def del_age(self):
```

```
    del self._age
```

→ # This deletes the attribute

`age = property(get_age, set_age, del_age)` → # The `property()` function binds these methods to the `age` attribute.

```
mark = GreekSC()
```

```
mark.age = 10
```

→ # The `age` attribute is accessed and set using the `property` function.

```
print(mark.age)
```

NOTES

3. Using @property decorators

→ In this approach, the `@property` decorator is used for the getter and the `@<property_name>.setter` decorator is used for the setter.

→ This approach allows a more elegant way to define getter and setter methods.

class Greets:

def __init__(self):

self.age = 0

using property decorator

a getter function

@property

def age(self):

print('getter method called')

return self.age

The Greets use the @property decorator for the getter method, which returns the age.

a setter function

@age.setter

def age(self):

if (a < 18):

raise ValueError("Sorry your age is below eligibility criteria")

print('setter method called')

self.age = a

The @age.setter decorator is used for the setter method, where a validation (age criteria) is added.

The setter is added to assign the value, while the getter retrieves it.

mark = Greets()

mark.age = 19

print(mark.age)

SUMMARY